

IOS · APP STORE CONNECT

Putting siteMargin on the App Store

The Android build is live on Google Play. This is the equivalent run for iOS — same app, a different, stricter pipeline. Steps are numbered in the order Apple actually forces on you.

Bundle ID `za.co.sitemargin.app`

Current repo: Android only, no `ios/` folder yet

Dev machine: Windows

Contents

00	Get access to a Mac	blocking
01	Enroll in the Apple Developer Program	\$99/yr
02	Add the iOS platform to the project	
03	App icon & splash screen for iOS	
04	Register the App ID & create the app record	
05	Build, sign, and archive	
06	Store listing & screenshots	
07	Privacy, age rating & review notes	
08	TestFlight	optional
09	Submit for review	

00 Get access to a Mac

Blocking · do this before anything else below

BLOCKER

Everything else on this page assumes Xcode. Xcode only runs on macOS — Apple doesn't offer a Windows path, unlike Android Studio. Your dev machine is Windows, so the whole iOS pipeline needs a Mac somewhere before Phase 2 can really start.

Claude can't drive a Mac the way it drove Android Studio — there's no Mac linked to this session. Whichever option below you pick, the actual clicking-through-Xcode part will need to happen with you at the machine, or through a CI service.

Rent one by the hour

MacinCloud or MacStadium give you a remote macOS desktop with Xcode preinstalled. Cheapest way to just get it done once.

Cloud build, no Mac at all

Capgo Build, Ionic Appflow, or Codemagic build and sign in their cloud, driven entirely from Windows.

Borrow or buy

A friend's Mac, an old Mac Mini, or a GitHub Actions macOS runner if you're comfortable scripting it.

Phase 5 below assumes Xcode directly, since that's the canonical path Apple's own docs describe — if you go the cloud-build route instead, the certificate/provisioning steps move into that service's dashboard, but everything from Phase 1–4 and 6–9 stays the same.

01 Enroll in the Apple Developer Program

\$99/year · 1 account covers every app you publish

☐ **Decide: Individual or Organization**

Individual is faster (24–48h verification) and lists you personally as the seller. Organization lists "siteMargin" / your registered company as the seller but needs a D-U-N-S number and takes 1–2 weeks. If siteMargin is a registered company and you want the company name on the listing, start the D-U-N-S lookup now — it alone takes 1–5 business days before Apple even begins reviewing you.

☐ **Enroll at developer.apple.com/programs/enroll**

Sign in with an Apple ID (create a dedicated one for the business if going the Organization route — don't use a personal one). Fill in the entity details, pay the \$99 fee.

☐ **Wait for verification**

You'll get an email once your account is active. Nothing else on this page is possible until this lands — App Store Connect, certificates, and the App ID all require an active paid membership.

02 Add the iOS platform to the project

On the Mac (or wherever Xcode lives) · in C:\Dev\site\margin if working locally first

☐ Install the iOS platform package

The project currently only has @capacitor/android — @capacitor/ios isn't installed yet.

```
npm install @capacitor/ios
```

☐ Build the web app and generate the native project

```
npm run build  
npx cap add ios  
npx cap sync ios
```

This creates a new ios/ folder with a real Xcode project (App.xcworkspace), reusing the same appId and appName already set in capacitor.config.json — no changes needed there.

☐ Commit the ios/ folder to the repo

Same as android/ — it's a real project folder that belongs in version control, not a build artifact.

03 App icon & splash screen for iOS

Different dimensions than Android — the Play Store assets can't be reused directly

☐ Prepare a 1024×1024 source icon with no transparency

Apple flat-out rejects an icon with an alpha channel — the existing `public/icon.png` is 1024×1024 RGBA (has alpha), so it needs flattening onto an opaque background before use, unlike the Android icon which was fine as-is.

☐ Prepare a 2732×2732 splash source

`@capacitor/assets` is already a dependency (used for Android) — it generates the full iOS icon and splash-screen image set from these two source files in one pass.

```
npx capacitor-assets generate --ios
```

04 Register the App ID & create the app record

developer.apple.com → Certificates, IDs & Profiles, then App Store Connect

☐ Register an explicit App ID

Certificates, IDs & Profiles → Identifiers → new App ID → Explicit → `za.co.sitemargin.app` (must match `capacitor.config.json` exactly). Leave capabilities off unless using push notifications or similar.

☐ Create the app record in App Store Connect

FIELD	VALUE
Platform	iOS
Name	siteMargin
Primary language	English (or English (UK) — matches en-ZA conventions better than en-US)
Bundle ID	za.co.sitemargin.app
SKU	sitemargin-ios-01

SKU is an internal reference only, never shown to users — anything unique works.

05 Build, sign, and archive

In Xcode, on the Mac

☐ **Open the workspace and set up signing**

Open `ios/App/App.xcworkspace` (not the `.xcodeproj`). Select the App target → Signing & Capabilities → tick "Automatically manage signing" → pick your team. Xcode mints a distribution certificate on first archive if you don't have one yet.

☐ **Back up the distribution certificate's private key**

Generated once and stored in the Mac's keychain only. If this Mac is a rental or borrowed machine, export it (Keychain Access → right-click the cert → Export) before losing access, or every future build needs a brand new certificate.

☐ **Archive**

Set the run destination to "Any iOS Device (arm64)" — not a simulator — then Product → Archive. Takes a few minutes.

☐ **Upload from the Organizer window**

When the archive finishes, the Organizer opens automatically → Distribute App → App Store Connect → Upload. Apple takes anywhere from a few minutes to about an hour to finish processing before it's selectable as a build.

06 Store listing & screenshots

App Store Connect → your app → the current (unreleased) version

☐ Fill in the text fields

Name, subtitle (30 characters, sits right under the name), promotional text, description, keywords, support URL, marketing URL. The short/full descriptions written for the Play Store listing carry over almost directly.

☐ Upload screenshots at exact native resolutions

Apple rejects anything off-spec — no cropping leeway like Play Store's 2:1 rule. At minimum, cover the 6.7" size; most teams also add 5.5" to satisfy older-device requirements. 3–10 images per size.

DEVICE	RESOLUTION	NEEDED?
6.7" iPhone (15/16 Pro Max class)	1290 × 2796	Required
5.5" iPhone (8 Plus class)	1242 × 2208	Recommended
iPad 12.9"	2048 × 2732	Only if the app supports iPad

If the app should be iPhone-only (matching the phone-only screenshots already made for Android), set that in Xcode: target → General → Supported Destinations → remove iPad. That also removes the iPad screenshot requirement here.

☐ Attach the uploaded build

Once Phase 5's upload finishes processing, it appears under the Build section on this same page — select it.

07 Privacy, age rating & review notes

The parts that don't exist on Google Play in the same shape

☐ **App Privacy — the "nutrition label"**

Apple's version of the Data Safety form already filled in for Play. Declares what's collected (account email, project/cost data tied to the user) and whether it's linked to identity — for siteMargin, this is data used to run the app itself, not shared with third parties or used for tracking.

☐ **Age rating questionnaire**

A short set of content questions (violence, gambling, user-generated content, etc.) — a business/productivity tool like this should land at 4+ straightforwardly.

☐ **Export compliance**

Asked on every upload. The app only uses standard HTTPS to talk to Supabase, so the answer is "Yes, uses encryption" → "Exempt" (standard/OS-provided encryption only) — no separate export documentation needed.

☐ **App Review information — leave a demo login**

Unlike Play's closed testing, Apple's human reviewer logs into the live app directly. Put working credentials in the "Notes" field of App Review Information — the same idea as the tester accounts set up for Android, or a dedicated review-only account so reviewers aren't touching real data.

☐ **Pricing and availability**

Free app with in-app purchases (matches the Contractor/Firm plans) → set price tier to Free, territories to South Africa or worldwide as preferred.

08 TestFlight — optional but worth it

Closest iOS equivalent to Play's closed testing track

☐ **Add internal testers**

Anyone with a role on the App Store Connect team gets an install automatically once a build is uploaded — no separate opt-in link needed, unlike Play.

☐ **Add external testers (optional)**

Up to 10,000 testers by email or a public link — the same tester emails used for Android would work here too. External testing requires one round of a lightweight Apple review first (usually same-day), separate from full App Store review.

09 Submit for review

The last button

☐ Add for Review, then Submit

Once every section above shows no warning icon, the "Add for Review" button becomes available at the top of the version page. Confirm, and it moves to "Waiting for Review."

☐ Wait for human review

Typically hours to a couple of days — slower and less automated than Play's ~14-minute check. If rejected, App Store Connect names the exact guideline and screen; most first-round rejections are metadata fixes, not code changes.

Sources: Capacitor — Deploying iOS Apps to the App Store (capacitorjs.com/docs/ios/deploying-to-app-store) · Josh Morony — Deploying Capacitor Applications to iOS (joshmorony.com) · Ionic — Building and Releasing Your Capacitor iOS App (ionic.io/blog) · MobileAction — App Store Screenshot Sizes & Guidelines (mobileaction.co) · Applefy — Apple Developer Program Enrollment (applefy.tech) · Capgo — Building an iOS App from Windows (capgo.app/blog)